S30 - EA2. Taller: Implementación de estructuras de datos

Franklin Darío Burbano Gaviria

Tecnología en Desarrollo de Software, IU Digital de Antioquia

Estructura de Datos PREICA2602B020075

Cristian David Lemus

8 de noviembre de 2026

1. Introducción

En el desarrollo de software, uno de los aspectos más importantes es encontrar la manera adecuada de organizar y administrar la información. No se trata solamente de almacenar datos, sino de hacerlo de forma que las operaciones de consulta, inserción y eliminación puedan realizarse eficientemente, especialmente cuando un sistema debe atender múltiples solicitudes.

A partir de esta necesidad, se plantea el caso de estudio de una cafetería denominada Coffee Shop, la cual requiere mejorar la administración de sus pedidos, productos y empleados. Para dar solución a esta situación, se propone desarrollar una aplicación utilizando Java y su entorno de colecciones (Java Collections Framework), aplicando los conocimientos adquiridos sobre estructuras de datos y programación orientada a objetos.

Durante el desarrollo se seleccionan tres estructuras principales: ArrayDeque para administrar los pedidos según su orden de llegada, HashSet para mantener un menú de productos sin elementos duplicados y HashMap para almacenar los empleados relacionando su código de identificación con su nombre.

La elección de estas estructuras responde a las características particulares de cada necesidad del negocio, considerando aspectos como la velocidad de las operaciones, la organización de la información y la facilidad de mantenimiento del código.

Con este ejercicio se busca llevar los conceptos teóricos a una situación práctica, comprendiendo que una buena selección de estructuras de datos contribuye a construir aplicaciones más organizadas, eficientes y preparadas para responder a las necesidades de sus usuarios.

2. Objetivos

2.1. Objetivo general

Desarrollar una solución informática para gestionar los pedidos, productos y empleados de una cafetería, mediante la implementación de estructuras de datos del Java Collections Framework, aplicando los principios de programación orientada a objetos y analizando la eficiencia de las operaciones realizadas.

2.2. Objetivos específicos

- Analizar los requerimientos del caso de estudio para identificar las estructuras de datos que mejor se adapten a cada necesidad.
- Implementar una cola mediante ArrayDeque que permita registrar y atender los pedidos respetando el orden de llegada.
- Utilizar HashSet para administrar los productos del menú, evitando registros duplicados y facilitando las consultas.

- Implementar HashMap para relacionar los identificadores únicos de los empleados con sus respectivos nombres.

- Aplicar conceptos de programación orientada a objetos, como clases, objetos, encapsulamiento, abstracción e interfaces.

- Comparar las estructuras seleccionadas teniendo en cuenta su funcionamiento y complejidad computacional.

3. Desarrollo

3.1. Análisis del problema

El caso de estudio presenta una cafetería que necesita organizar tres procesos fundamentales para su funcionamiento: la recepción de pedidos, la administración de los productos disponibles y la consulta de los empleados.

Aunque estos procesos están relacionados, cada uno tiene necesidades diferentes. Por esta razón, no sería conveniente utilizar una misma estructura de datos para administrar toda la información. Es necesario seleccionar una alternativa que responda adecuadamente a las operaciones que se realizan en cada módulo.

3.2. Gestión de pedidos mediante ArrayDeque

Para administrar los pedidos se seleccionó la estructura ArrayDeque, utilizando la interfaz Queue del Java Collections Framework.

Esta elección se debe a que los pedidos deben atenderse respetando estrictamente el orden en que fueron recibidos. Este comportamiento corresponde al principio FIFO (First In, First Out), que significa que el primer elemento en ingresar es también el primero en salir.

En la aplicación, los pedidos se incorporan al final de la cola mediante el método offer() y se retiran desde el inicio utilizando poll(). También se puede consultar el siguiente pedido mediante peek(), sin necesidad de retirarlo.

Esta estructura permite realizar las operaciones en los extremos con una complejidad O(1) amortizada, por lo que resulta apropiada para el procesamiento continuo de pedidos.

3.3. Administración del menú mediante HashSet

Para almacenar los productos disponibles se implementó un HashSet, a través de la interfaz Set.

La razón principal de esta elección es que el menú debe contener productos únicos. Por ejemplo, si se intenta registrar dos veces el producto Latte, el conjunto evita que se almacene un elemento duplicado.

Para comprobar si un producto está disponible se utiliza el método contains(), mientras que add() permite incorporar nuevos productos y remove() facilita su eliminación.

HashSet ofrece un rendimiento promedio O(1) en las operaciones de inserción, eliminación y búsqueda, lo que permite consultar los productos de manera eficiente sin necesidad de mantenerlos ordenados.

3.4. Registro de empleados mediante HashMap

Para el registro de empleados se seleccionó HashMap, implementado mediante la interfaz Map.

Esta estructura permite almacenar información en parejas de clave y valor. En este caso, el código de identificación del empleado funciona como clave, mientras que su nombre corresponde al valor asociado.

Por ejemplo, el identificador 101 puede estar relacionado con el empleado Carlos. Cuando el sistema necesita consultar un empleado, utiliza el método get() y proporciona su código de identificación.

También se utiliza containsKey() para verificar si el identificador se encuentra registrado.

HashMap presenta una complejidad promedio O(1) para las operaciones de consulta, inserción y eliminación por clave, lo que resulta conveniente para el acceso rápido a los datos de los empleados.

3.5. Aplicación de la programación orientada a objetos

Durante el desarrollo se aplicaron diferentes principios de la programación orientada a objetos para mantener el código organizado y facilitar su comprensión.

Se utilizaron clases para representar las entidades del sistema, como los pedidos y los empleados. También se definieron atributos para almacenar sus características y métodos para ejecutar las operaciones correspondientes.

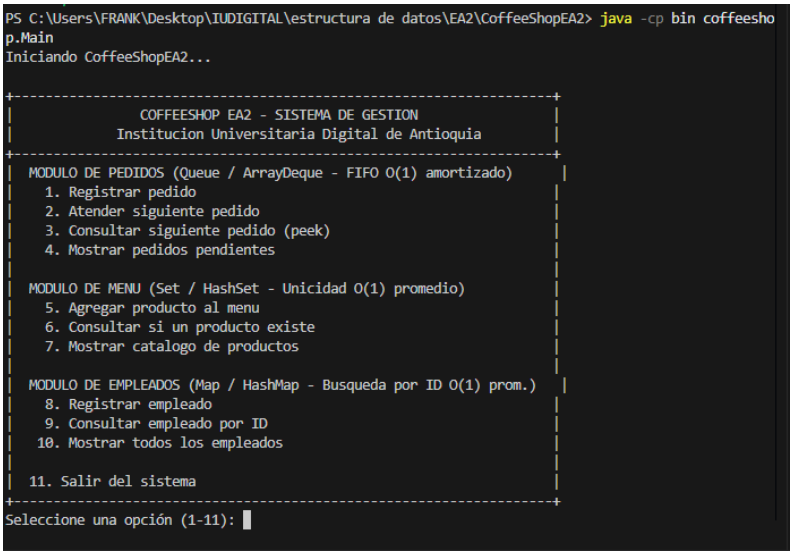
El encapsulamiento permite controlar el acceso a los datos, mientras que la abstracción facilita trabajar con las funcionalidades de cada módulo sin depender directamente de todos sus detalles internos.

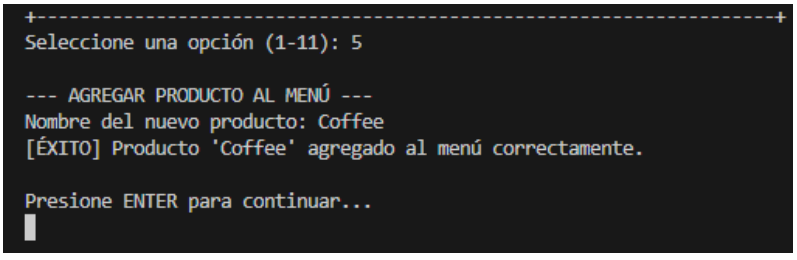
Adicionalmente, se emplearon interfaces como Queue, Set y Map, asociadas a sus implementaciones concretas ArrayDeque, HashSet y HashMap. Esta forma de trabajar permite mantener una mayor flexibilidad en el código y facilita posibles cambios en las implementaciones.

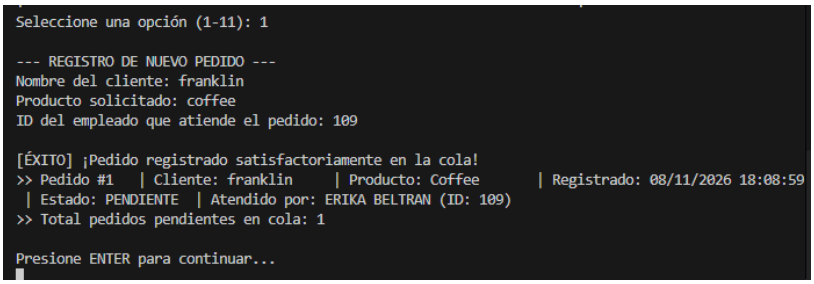
3.6. Comparación de las estructuras implementadas

Necesidad	Estructura	Operación principal	Rendimiento
Registro de pedidos	ArrayDeque	offer() y poll()	O(1) amortizado
Menú de productos	HashSet	contains()	O(1) promedio
Registro de empleados	HashMap	get()	O(1) promedio

La comparación permite observar que cada estructura responde a un propósito específico. La cola conserva el orden de procesamiento, el conjunto evita duplicados y el mapa facilita la recuperación de información mediante identificadores.







Anexo video. https://drive.google.com/file/d/1xzmVMf-0Rmv6C07DpXrHyrbJXURizkW1/view?usp=sharing

4. Conclusiones

El desarrollo del caso Coffee Shop permitió comprender que la selección de una estructura de datos no debe realizarse de manera arbitraria, sino teniendo en cuenta las operaciones que necesita ejecutar el sistema y la forma en que se debe organizar la información.

La implementación de ArrayDeque permite representar adecuadamente el proceso de atención de pedidos, ya que conserva el orden de llegada y facilita la incorporación y extracción de elementos. Por su parte, HashSet ofrece una solución práctica para administrar los productos del menú sin permitir duplicados, mientras que HashMap facilita la consulta de los empleados mediante su identificador único.

Asimismo, el ejercicio permitió reconocer la importancia de la programación orientada a objetos y del uso de interfaces en Java. Estos elementos ayudan a desarrollar programas con una

mejor separación de responsabilidades, mayor flexibilidad y una estructura más sencilla de mantener.

Finalmente, se puede concluir que una adecuada combinación de estructuras de datos contribuye a mejorar el rendimiento de una aplicación y a resolver necesidades concretas de un negocio. El caso estudiado demuestra cómo los conocimientos teóricos sobre TDA y Java Collections Framework pueden aplicarse en la construcción de soluciones informáticas funcionales.